

РЕФЕРАТ

Курсовой проект содержит 40 страниц, 8 таблиц, 11 рисунков, 8 использованных источников, 2 приложения.

Ключевые слова: БИНАРНОЕ ДЕРЕВО, МАССИВЫ ДАННЫХ, МЕТОД СОРТИРОВКИ, ПРОГРАММА, АЛГОРИТМ, СОРТИРОВКА, ДАННЫЕ.

Целью данной работы является проектирование и разработка программы для работы с кольцевым списком с использованием языка C# и Visual Studio 2019.

Результат работы: получение знаний в области структур данных и их реализации, алгоритмов для решения поставленной задачи.

В данной работе представлены алгоритмы для реализации сортировки, также представлены алгоритмы для сравнения эффективности работы алгоритма с бинарным деревом. Предоставлена инструкция пользователя по данной программе.

Объект исследования: бинарное дерево.

Предмет исследования: программа, написанная на языке C#.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1 ОПИСАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ	7
1.1 Общие положения	7
1.2 Сведения из теории	7
1.2.1 Древовидная структура – «Бинарное дерево»	7
1.2.2 Сортировка бинарным деревом	8
1.2.3 Терминология	8
1.2.4 Обход бинарного дерева в глубину	9
1.3 Постановка задачи	9
2 ТЕХНОЛОГИЯ РАЗРАБОТКИ ПРОГРАММЫ	11
2.1 Алгоритм решения	11
2.2 Макет программы	12
2.3 Описание программы	13
2.4 Результат работы программы	14
2.5 Сравнительный анализ	17
3 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ	21
ЗАКЛЮЧЕНИЕ	23
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	24
ПРИЛОЖЕНИЕ	25

ВВЕДЕНИЕ

Данная работа посвящена исследованию и анализу методов сортировок структур данных. Методы сортировки, рассматриваемые в данном курсовом проекте, будут анализироваться и сравниваться в эффективности использования в том или ином случае применения.

Сортировка подразумевает под собой чётко построенный порядок элементов данных. С точки зрения программирования, сортировка – алгоритм, который перебирает элементы данных по каким-либо параметрам, указываемых при построении данного алгоритма поиска элементов. С точки зрения информатики, сортировка – структура, содержащая в себе чётко структурированные данные для дальнейшей обработки в информацию.

Метод сортировки с помощью бинарного дерева - метод, рассматриваемый в данной работе, будет сравнен с алгоритмами сортировки пузырьком и алгоритмом гномьей сортировки для наглядного анализа эффективности работы алгоритма.

Для сравнения алгоритмов сортировки можно использовать следующие критерии: время работы алгоритма, количество перестановок элементов, объём занимаемой оперативной памяти компьютера, загруженность процессора компьютера, количество обработанных данных в единицу времени, способность повторять одинаковые значения при количественных прогонах алгоритма. В данной работе будет использовано два критерия сравнения – это время работы алгоритма в тиках и количество перестановок элементов.

Алгоритм бинарного дерева был изобретён в 1960 г. Андрю Дональдсом Ботом. На методе бинарного дерева построена файловая система, которой каждый из нас пользуется в компьютере, смартфоне. Структура бинарного дерева лучше всего подходит для реализации «Проводника», то есть «папка-подпапка» или правильнее говоря «узел-

поддереву».

Цель данного курсового проекта – это разработка программы для анализа методов сортировки в среде Microsoft Visual Studio 2019.

Задачи:

1. Разработать и доступно описать предметную область программы;
2. Разобрать алгоритм работы программы;
3. Продемонстрировать как реализована программа и протестировать её.

1 ОПИСАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Общие положения

Целью данной работы является разработка программы для исследования методов сортировки с помощью деревьев в среде Microsoft Visual Studio 2019.

1.2 Сведения из теории

1.2.1 Древовидная структура – «Бинарное дерево»

Бинарное (двоичное) дерево – это упорядоченное дерево, каждая вершина которого имеет не более двух поддеревьев, причем для каждого узла выполняется правило: в левом поддереве содержатся только ключи, имеющие значения, меньшие, чем значение данного узла, а в правом поддереве содержатся только ключи, имеющие значения, большие, чем значение данного узла.

Бинарное дерево является рекурсивной структурой, поскольку каждое его поддерево само является бинарным деревом и, следовательно, каждый его узел в свою очередь является корнем дерева.

Узел дерева, не имеющий потомков, называется листом.

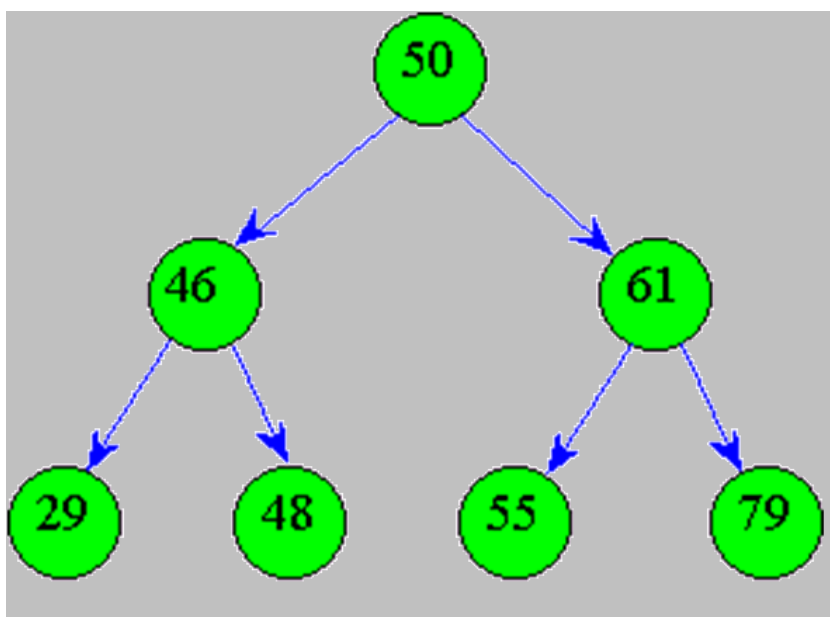


Рис.1 – «Бинарное дерево»

Глубина бинарного дерева – это максимальный уровень листа дерева, иначе говоря, длина самого длинного пути от корня к листу дерева.

Возможны случаи, когда бинарное дерево может представлять собой пустое множество или бинарное дерево может вырождаться в список, в данном случае такое дерево будет называться вырожденным.

Бинарное дерево будет представлять собой пустое множество, только в случаях отсутствия корня.

1.2.2 Сортировка бинарным деревом

Сортировка бинарным деревом (Tree sort) – алгоритм сортировки, который заключается в построении двоичного дерева поиска по ключам массива, с последующим построением результирующего массива упорядоченных элементов путём обхода дерева.

1.2.3 Терминология

Терминология, применяемая для описания бинарных деревьев:

1. узел – это точка, где может возникнуть ветвь;
2. корень – «верхний» узел дерева;
3. ветвь – отрезок, описывающий связь между двумя узлами;
4. лист – узел, из которого не выходят ветви, то есть не имеющий поддеревьев;
5. родительским называется узел, который находится непосредственно над другим узлом;
6. дочерним – называется узел, который находится непосредственно под другим узлом;
7. предки данного узла – это все узлы на пути вверх от данного узла до корня;
8. потомки – все узлы, расположенные ниже данного;
9. внутренний узел – узел, не являющийся листом;

- 10.порядок узла – количество его дочерних узлов;
- 11.глубина узла – количество его предков плюс единица;
- 12.глубина (высота) дерева – максимальная глубина всех узлов;
- 13.длина пути к узлу – количество ветвей, которые нужно пройти, чтобы дойти от корня к данному узлу;
- 14.длина пути дерева (длина внутреннего пути) – сумма длин путей всех его узлов.

1.2.4 Обход бинарного дерева в глубину

Существуют три порядка, использующие обход в глубину – PreOrder, InOrder, PosOrder.

1. Сверху вниз (PreOrder):
 - a. Обработать корень;
 - b. Обход левого поддерева;
 - c. Обход правого поддерева;
2. Слева направо (InOrder):
 - a. Обход левого поддерева;
 - b. Обработать корень;
 - c. Обход правого поддерева;
3. Снизу вверх (PosOrder):
 - a. Обход левого поддерева;
 - b. Обход правого поддерева;
 - c. Обработать корень.

1.3 Постановка задачи

Написать программу для исследования методов сортировки с помощью деревьев в среде Microsoft Visual Studio 2019, реализовать возможность сравнения с другими методами сортировок, создать удобный пользовательский интерфейс.

Название: «Методы сортировки»

Назначение: исследование методов сортировок, сравнительный анализ, выявление эффективности.

Для разработки программы выбран объектно-ориентированный язык программирования C#. Он хорошо организован, строг, большинство его конструкций логичны и удобны. Немаловажно, что C# является профессиональным языком, предназначенным для решения широкого спектра задач, и в первую очередь - в быстро развивающейся области создания распределенных приложений. Язык имеет статическую типизацию, поддерживает полиморфизм, перегрузку операторов (в том числе операторов явного и неявного приведения типа), делегаты, атрибуты, события, свойства, обобщённые типы и методы, итераторы, анонимные функции с поддержкой замыканий, LINQ, исключения, комментарии в формате XML.

Выбор среды разработки обоснован тем, что Microsoft Visual Studio – одно из лучших решений для разработки приложений на языке C#, это передовая среда разработки на языке C#, предназначенная для создания интерактивных приложений с пользовательским интерфейсом для компьютеров, рабочих станций, сенсорных дисплеев, информационных терминалов и Интернета. На сегодняшний день это довольно мощная среда программирования, она обладает богатым функционалом, быстро работает, занимает мало места.

2 ТЕХНОЛОГИЯ РАЗРАБОТКИ ПРОГРАММЫ

2.1 Алгоритм решения

В данной работе были применены такие алгоритмы как:

1. Алгоритм гномьей сортировки (Gnome Sort);
2. Алгоритм сортировки пузырьком (Bubble Sort);
3. Алгоритм сортировки с помощью бинарных деревьев (Tree Sort).

Гномья сортировка (Gnome Sort) – простой в реализации алгоритм сортировки массива, назван в честь садового гнома, который предположительно таким методом сортирует садовые горшки.

Принцип работы алгоритма сортировки Гнома: алгоритм находит первую пару неотсортированных элементов массива и меняет их местами. При этом учитывается факт, что обмен приводит к неправильному расположению элементов, примыкающих с обеих сторон к только что переставленным. Поскольку все элементы массива после переставленных не отсортированы, необходимо перепроверить только элементы до переставленных.

Сортировка пузырьком (Bubble Sort) - один из самых простых для понимания методов сортировки массивов.

Описание алгоритма сортировки пузырьком: алгоритм заключается в циклических проходах по массиву, за каждый проход элементы массива попарно сравниваются и, если их порядок не правильный, то осуществляется обмен. Обход массива повторяется до тех пор, пока массив не будет упорядочен.

Сортировка бинарным деревом (Tree Sort) – алгоритм сортировки, который заключается в построении двоичного дерева поиска по ключам массива, с последующим построением результирующего массива упорядоченных элементов путем обхода дерева.

Принцип работы алгоритма сортировки бинарным деревом:

1. Элементы неотсортированного массива данных добавляются в

двоичное дерево поиска;

2. Для получения отсортированного массива, производится обход бинарного дерева с переносом данных из дерева в результирующий массив.

2.2 Макет программы

Для удобства пользования программой был спроектирован пользовательский интерфейс, включающий в себя такие элементы как поле начального массива, поля сортировок: гномья, пузырьковая и деревом. Также поля для вывода данных о работах алгоритмов, таких показателей как время работы и количество перестановок.

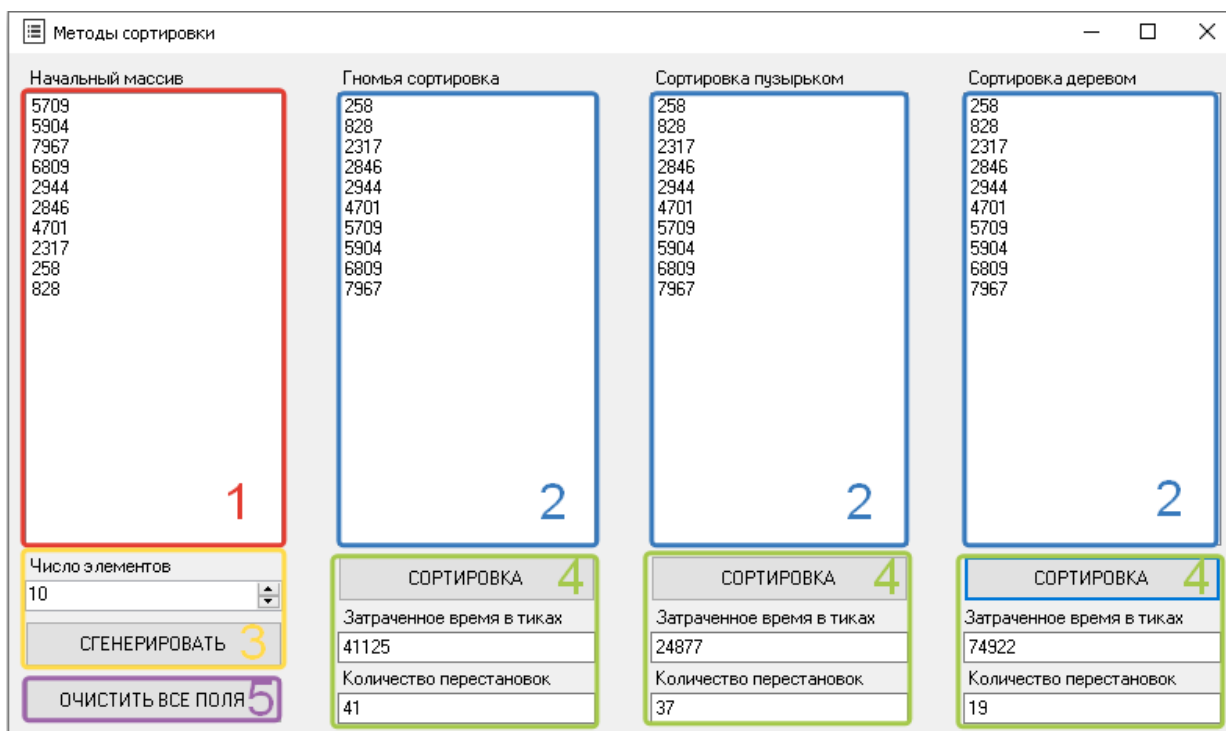


Рис. 2 – Макет программы «Методы сортировок».

На Рис. 2 отображён макет программы «Методы сортировок», где каждой цифре на рисунке соответствует свой функционал:

1. Блок с полем вывода для визуального представления генерируемого массива для дальнейшей работы алгоритмов

сортировок;

2. Блоки с полями вывода для визуального представления отсортированного изначального массива, который представлен в блоке 1;
3. Блок, включающий в себя поле ввода размера массива, над которым будут производиться алгоритмические действия, и функциональную кнопку для генерирования массива с заданным числом элементов;
4. Блоки, включающие в себя функциональную кнопку для запуска алгоритма сортировки, также поля для вывода сравнительных показателей работы алгоритма;
5. Блок с кнопкой, предназначенной для очистки всех полей программы.

2.3 Описание программы

Программа имеет пользовательский интерфейс, и пользователь может пользоваться программой с помощью кнопок, которые будут описаны ниже.

Для корректной работы алгоритмов необходим глобальный массив, который в коде объявлен как `array`; и необходим метод `Stopwatch` для дальнейшего вывода времени работы алгоритмов сортировок.

Также для оптимизации памяти была вынесена глобальная функция `Swap`, которая используется для двух кнопок. Функция `Swap`, меняет элементы местами, также здесь прописан счётчик для сравнительного анализа алгоритмов.

Программа имеет класс `TreeNode`, где находится код реализации бинарного дерева. Класс `TreeNode` включает в себя две функции: `Insert` и `Transform` – функции вставки и перемещения.

Под кнопкой `button1` происходит считывание количества элементов для массива, а также заполнение данного массива с помощью метода

Random и вывода полученного массива в поле listBox.

Под кнопкой button2 находится алгоритм гномьей сортировки, который включает в себя функцию GnomeSort.

Под кнопкой button3 находится алгоритм сортировки пузырьком, который включает в себя функцию BubbleSort.

Под кнопкой button4 находится алгоритм сортировки с помощью бинарного дерева, включающий в себя функцию TreeSort.

Под кнопкой button5 находятся команды очистки всех имеющихся полей в форме, обнуление поля numericUpDown и сброс счётчика времени.

Код программы представлен в Приложении 1. Блок-схема программы представлена в Приложении 2.

2.4 Результат работы программы

Результатом работы программы является вывод в поля заданного количественно отсортированного массива с помощью трёх методов сортировки: гномьей сортировки, сортировки пузырьком и сортировкой бинарным деревом.

Методы сортировки

Начальный массив

Гномья сортировка

Сортировка пузырьком

Сортировка деревом

Число элементов
0

СГЕНЕРИРОВАТЬ

ОЧИСТИТЬ ВСЕ ПОЛЯ

СОРТИРОВКА

Затраченное время в тиках

Количество перестановок

СОРТИРОВКА

Затраченное время в тиках

Количество перестановок

СОРТИРОВКА

Затраченное время в тиках

Количество перестановок

Рис. 3 – Начало работы программы.

Были задействованы поле ввода числа элементов и кнопки «СГЕНЕРИРОВАТЬ» для количественно генерирования массива элементов.

Рис. 4 – Задание количественно генерируемого массива.

При нажатии кнопок «СОРТИРОВКА» выводятся в поля отсортированный массив, затраченное время в тиках и количество перестановок.

Методы сортировки

Начальный массив

Гномья сортировка

Сортировка пузырьком

Сортировка деревом

Число элементов: 100

СГЕНЕРИРОВАТЬ

ОЧИСТИТЬ ВСЕ ПОЛЯ

СОРТИРОВКА

Затраченное время в тиках: 602317

Количество перестановок: 2708

СОРТИРОВКА

Затраченное время в тиках: 567429

Количество перестановок: 2704

СОРТИРОВКА

Затраченное время в тиках: 572603

Количество перестановок: 199

Рис.5 – Результат работы программы.

При нажатии на кнопку «ОЧИСТИТЬ ВСЕ ПОЛЯ» очистятся все поля.

Методы сортировки

Начальный массив

Гномья сортировка

Сортировка пузырьком

Сортировка деревом

Число элементов: 0

СГЕНЕРИРОВАТЬ

ОЧИСТИТЬ ВСЕ ПОЛЯ

СОРТИРОВКА

Затраченное время в тиках:

Количество перестановок:

СОРТИРОВКА

Затраченное время в тиках:

Количество перестановок:

СОРТИРОВКА

Затраченное время в тиках:

Количество перестановок:

Рис. 6 – Очищенные поля.

2.5 Сравнительный анализ

В ходе данной работы был проведен сравнительный анализ работы алгоритмов сортировки по таким критериям – затраченное время в тиках и количество перестановок.

Таблица 1 – «Гномья сортировка».

гномья сортировка	размер массива				
	10	100	1000	5000	10000
затраченное время в тиках	46992	987503	10302682	55241882	110770061
количество перестановок	19	2845	253032	6159617	25178960



Рис. 7 – «Гномья сортировка».

Таблица 2 – «Сортировка пузырьком».

сортировка пузырьком	размер массива				
	10	100	1000	5000	10000
затраченное время в тиках	44692	994008	10491332	58088653	116343018
количество перестановок	16	2837	252986	6158329	25174039

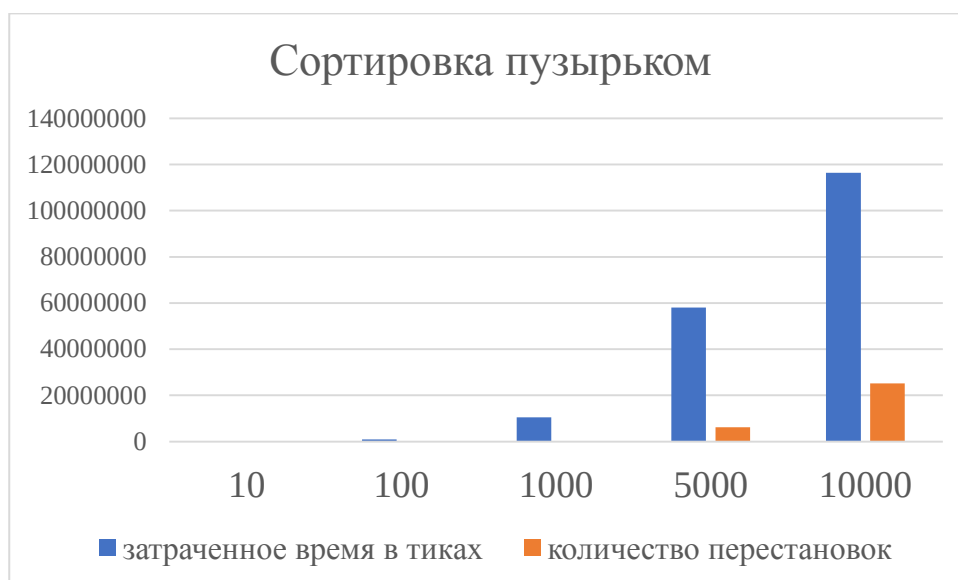


Рис. 8 - «Сортировка пузырьком».

Таблица 3 – «Сортировка деревом».

сортировка деревом	размер массива				
	10	100	1000	5000	10000
затраченное время в тиках	42528	981878	10296626	52321915	97745791
количество перестановок	19	199	1999	9999	19999

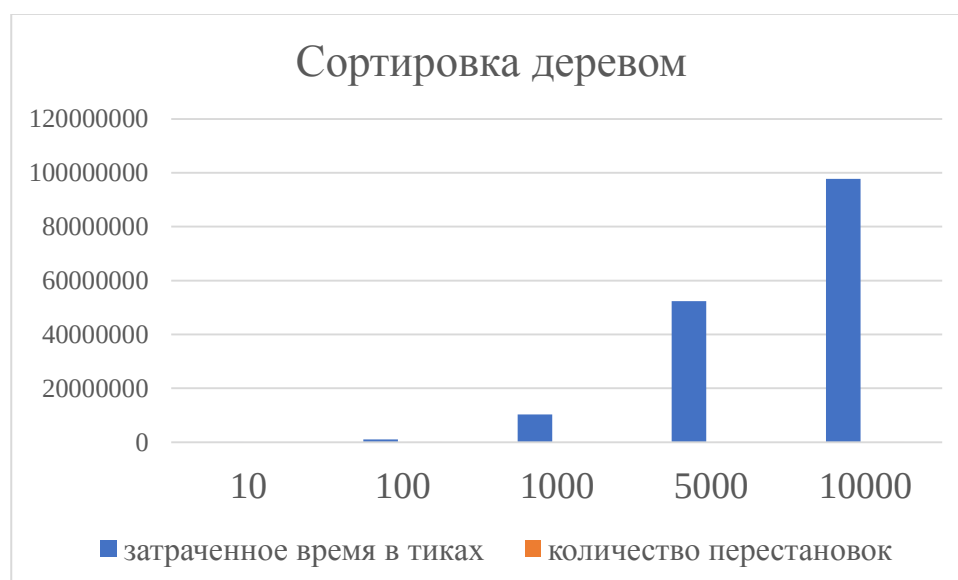


Рис. 9 - «Сортировка деревом».

Таблица 4 – «Сравнительная таблица 10 элементов».

размер массива 10	гномья сортировка	сортировка пузырьком	сортировка деревом
затраченное время в тиках	46992	44692	42528
количество перестановок	19	16	19

Таблица 5 – «Сравнительная таблица 100 элементов».

размер массива 100	гномья сортировка	сортировка пузырьком	сортировка деревом
затраченное время в тиках	987503	994008	981878
количество перестановок	2845	2837	199

Таблица 6 – «Сравнительная таблица 1000 элементов».

размер массива 1000	гномья сортировка	сортировка пузырьком	сортировка деревом
затраченное время в тиках	10302682	10491332	10296626
количество перестановок	253032	252986	1999

Таблица 7 – «Сравнительная таблица 5000 элементов».

размер массива 5000	гномья сортировка	сортировка пузырьком	сортировка деревом
затраченное время в тиках	55241882	58088653	52321915
количество перестановок	6159617	6158329	9999

Таблица 8 – «Сравнительная таблица 10000 элементов».

размер массива 10000	гномья сортировка	сортировка пузырьком	сортировка деревом
затраченное время в тиках	110770061	116343018	97745791
количество перестановок	25178960	25174039	19999

Из приведённых выше таблиц можно сделать вывод, что при увеличении объёма изначально генерируемого массива самым быстрым способом сортировки оказался метод сортировки бинарным деревом, значительно превосходя гномью сортировку и сортировку пузырьком.

Сравнивая показатели перестановок, можно сделать вывод, что сортировка методом бинарного дерева в разы эффективнее гномьей сортировки и сортировки пузырьком. При этом алгоритмы гномьей сортировки и сортировки пузырьком используют одну и ту же функцию Swap, а значения их показателей отличаются лишь незначительно из-за особенностей своих алгоритмов сортировок.

3 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

После запуска программы появляется окно, как показано на Рис. 10.

Рис. 10 – Первый запуск программы.

Для начала работы программы необходимо задать количество элементов в поле «Число элементов» и нажать кнопку «СГЕНЕРИРОВАТЬ».

В поле «Начальный массив» появится массив из заданного числа элементов.

Чтобы начать сортировку каким-либо методом, необходимо нажать кнопку «СОРТИРОВКА», при этом может потребоваться некоторое время для обработки массива. После отсортированного массив выведется в поле соответствующей кнопке и в поля выведется «Затраченное время в тиках» и «Количество перестановок» значения соответственно.

Чтобы очистить все поля в программе необходимо нажать кнопку «ОЧИСТИТЬ ВСЕ ПОЛЯ», после чего все поля будут пусты.

На Рис. 11 показан результат выполнения программы.

Начальный массив	Гномья сортировка	Сортировка пузырьком	Сортировка деревом
5141 2051 99 7238 7398 8725 722 5093 998 6207 2860 9926 9101 3412 7504 9166 1082 8343 7947 1395 1389 522	85 99 115 238 522 722 723 734 859 877 945 998 1082 1124 1140 1168 1190 1389 1395 1401 1584 1617	85 99 115 238 522 722 723 734 859 877 945 998 1082 1124 1140 1168 1190 1389 1395 1401 1584 1617	85 99 115 238 522 722 723 734 859 877 945 998 1082 1124 1140 1168 1190 1389 1395 1401 1584 1617
Число элементов 100	СОРТИРОВКА	СОРТИРОВКА	СОРТИРОВКА
СГЕНЕРИРОВАТЬ	Затраченное время в тиках 602317	Затраченное время в тиках 567429	Затраченное время в тиках 572603
ОЧИСТИТЬ ВСЕ ПОЛЯ	Количество перестановок 2708	Количество перестановок 2704	Количество перестановок 199

Рис. 11 – Результат работы программы.

Пользователь не может ввести буквы вместо цифр в поле «Число элементов», в нём заданы только числовые значения, но максимальное число, которое может ввести пользователь равно одному миллиону.

В поля «Затраченное время в тиках» и «Количество перестановок» пользователь может как буквы, так и цифры, но при нажатии на кнопку «СОРТИРОВКА» поля очищаются и в них выводятся значения.

ЗАКЛЮЧЕНИЕ

Целью работы являлась разработка программы для исследования методов сортировки с помощью деревьев в среде разработки Microsoft Visual Studio 2019, цель достигнута полностью. В качестве инструмента использовалась Microsoft Visual Studio 2019. Задачи курсового проекта, разработка и тестирование были решены, во время выполнения поставленной задачи были улучшены навыки программирования. В ходе работы была написана программа, задачей которой было реализация методов сортировки с помощью деревьев. Для решения задачи были изучены методы сортировок, теоретический материал.

При выполнении курсового проекта произведено углубление в информационные источники по объектно-ориентированному программированию с целью анализа состояния решаемой задачи.

Был проведен анализ предметной области, выявлены требования к разрабатываемой программе, было спроектировано и реализовано приложение, определена эффективность разработки.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. «Никлаус Вирт: Алгоритмы и структуры данных. Новая версия для Оберона.» Справочное пособие/Никлаус Вирт – М: издательство «ДМК-Пресс», 2 издание, 2016 г.
2. Алгоритмы и структуры данных – URL: <http://aisd-kubsau0.1gb.ru/> (Дата обращения 14.12.2019)
3. Джозеф Албахари, Бен Албахари. Полное описание языка C# 5.0. Справочное пособие/Джозеф Албахари, Бен Албахари. – М: издательство «Вильямс», 5 издание, 2014 г.
4. Джон Скит «C# для профессионалов: тонкости программирования.» Справочное пособие/Джон Скит - 3-е издание. – М: «Вильямс», 2014 г.
5. Эндрю Троелсен «Язык программирования C# 5.0 и платформа .NET 4.5». Справочное пособие/Эндрю Троелсен.– М: Издательство «Вильямс», 2013 г.
6. Годштейн Саша - Оптимизация приложений на платформе .NET. – М: «ДМК Пресс», 2014 г.
7. Павловская Татьяна - C#. Программирование на языке высокого уровня СПб: «Питер», 2014 г.
8. Бондарев В.М., Рублинецкий В.И., Качко Е.Г. Основы программирования. —Харьков: Фолио, Ростов н/Д: Феникс, 2014 г.

ПРИЛОЖЕНИЕ

Приложение 1: Код программы.

```
using System;
using System.Collections.Generic;
using System.Windows.Forms;
using System.Diagnostics;
namespace Курсовой_проект_Максименко_ИТ1823
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
        }
        public int[] array;//объявление глобального массива
        Stopwatch time = new Stopwatch();
        private void button1_Click(object sender, EventArgs e)//начальный массив
        {
            listBox.Items.Clear();
            int n = Convert.ToInt32(numericUpDown.Value.ToString());
            Random rnd = new Random();
            array = new int[n];
            for (int i = 0; i < array.Length; i++)
            {
                array[i] = rnd.Next(0, 10000);
                listBox.Items.Add(array[i]);
            }
        }
    }
}
```



```

private void button2_Click(object sender, EventArgs e)//Гномья сортировка
{
    listBox1.Items.Clear();
    textBox1.Clear();
    textBox4.Clear();
    time.Reset();
    time.Start();
    ulong perestanovka = 0;
    int[] array1 = new int[array.Length];
    for (int i = 0; i < array1.Length; i++)
    {
        array1[i] = Convert.ToInt32(array[i]);
    }
    //Гномья сортировка
    int[] GnomeSort(int[] unsortedArray)
    {
        var index = 1;
        var nextIndex = index + 1;
        while (index < unsortedArray.Length)
        {
            if (unsortedArray[index - 1] < unsortedArray[index])
            {
                index = nextIndex;
                nextIndex++;
            }
            else
            {
                Swap(ref unsortedArray[index - 1], ref unsortedArray[index], ref
perestanovka);
                index--;
            }
        }
    }
}

```

```

        if (index == 0)
        {
            index = nextIndex;
            nextIndex++;
            perestanovka++;
        }
    }
}

return unsortedArray;
}

//вывод отсортированного массива
array1 = GnomeSort(array1);
for (int i = 0; i < array1.Length; i++)
{
    listBox1.Items.Add(array1[i]);
}

time.Stop();
textBox1.Text = Convert.ToString(time.ElapsedTicks);
textBox4.Text = Convert.ToString(perestanovka);
}

private void button3_Click(object sender, EventArgs e)//сортировка
пузырьком
{
    listBox2.Items.Clear();
    textBox2.Clear();
    textBox5.Clear();
    time.Reset();
    time.Start();
    ulong perestanovka = 0;
    int[] array2 = new int[array.Length];

```

```

for (int i = 0; i < array2.Length; i++)
{
    array2[i] = Convert.ToInt32(array[i]);
}
//сортировка пузырьком
int[] BubbleSort(int[] array)
{
    var len = array.Length;
    for (var i = 1; i < len; i++)
    {
        for (var j = 0; j < len - i; j++)
        {
            if (array[j] > array[j + 1])
            {
                Swap(ref array[j], ref array[j + 1], ref perestанovka);
            }
        }
    }
    return array;
}
//вывод отсортированного массива
array2 = BubbleSort(array2);
for (int i = 0; i < array2.Length; i++)
{
    listBox2.Items.Add(array2[i]);
}
time.Stop();
textBox2.Text = Convert.ToString(time.ElapsedTicks);
textBox5.Text = Convert.ToString(perestанovka);
}

```

```
private void button4_Click(object sender, EventArgs e)//сортировка  
деревом
```

```
{  
    listBox3.Items.Clear();  
    textBox3.Clear();  
    textBox6.Clear();  
    time.Reset();  
    time.Start();  
    ulong perestанovka = 0;  
    int[] array3 = new int[array.Length];  
    for (int i = 0; i < array3.Length; i++)  
    {  
        array3[i] = Convert.ToInt32(array[i]);  
    }  
    //метод для сортировки с помощью двоичного дерева  
    int[] TreeSort(int[] array)  
    {  
        var treeNode = new TreeNode(array[0]);  
        for (int i = 1; i < array.Length; i++)  
        {  
            treeNode.Insert(new TreeNode(array[i]));  
            perestанovka++;  
        }  
        return treeNode.Transform(ref perestанovka);  
    }  
    //вывод отсортированного массива  
    array3 = TreeSort(array3);  
    for (int i = 0; i < array3.Length; i++)  
    {  
        listBox3.Items.Add(array3[i]);  
    }  
}
```

```

    }
    time.Stop();
    textBox3.Text = Convert.ToString(time.ElapsedTicks);
    textBox6.Text = Convert.ToString(perestanovka);
}
public class TreeNode//простая реализация бинарного дерева
{
    public TreeNode(int data)
    {
        Data = data;
    }
    public int Data { get; set; }//данные
    public TreeNode Left { get; set; }//левая ветка дерева
    public TreeNode Right { get; set; }//правая ветка дерева
    public void Insert(TreeNode node)//рекурсивное добавление узла в
дерево
    {
        if (node.Data < Data)
        {
            if (Left == null)
            {
                Left = node;
            }
            else
            {
                Left.Insert(node);
            }
        }
        else
        {

```

```

        if (Right == null)
        {
            Right = node;
        }
        else
        {
            Right.Insert(node);
        }
    }
}

public int[] Transform(ref ulong count, List<int> elements =
null)//преобразование дерева в отсортированный массив
{
    if (elements == null)
    {
        elements = new List<int>();
    }
    if (Left != null)
    {
        Left.Transform(ref count, elements);
    }
    elements.Add(Data);
    if (Right != null)
    {
        Right.Transform(ref count, elements);
    }
    count++;
    return elements.ToArray();
}
}

```

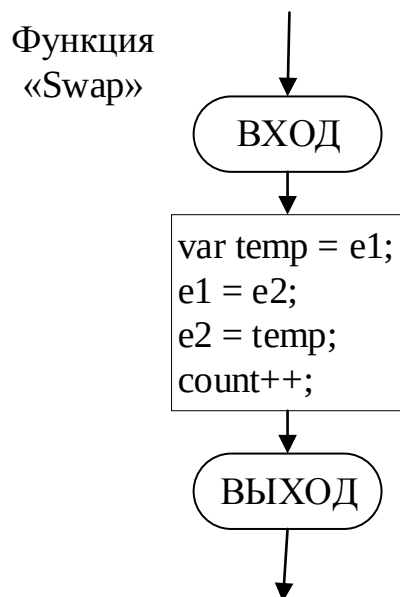
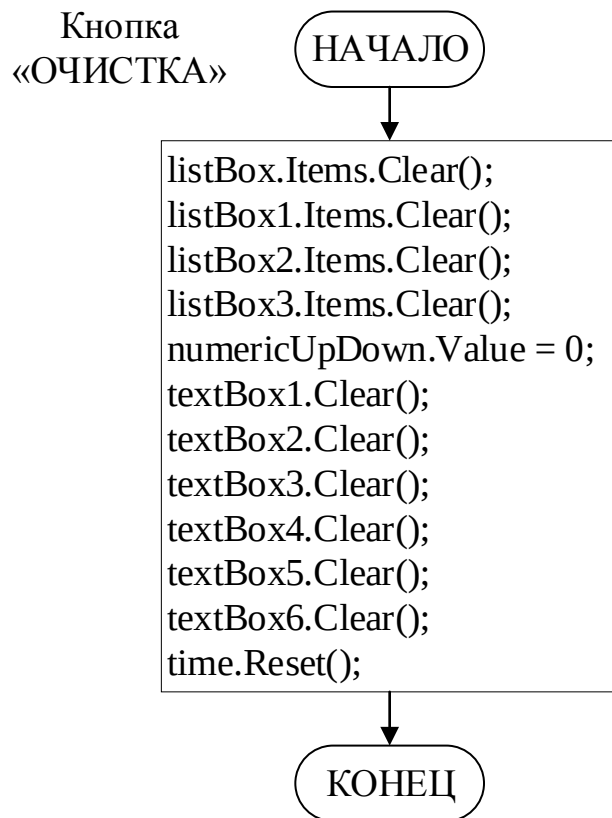
```

    public void Swap(ref int e1, ref int e2, ref ulong count)//метод обмена
элементов
    {
        var temp = e1;
        e1 = e2;
        e2 = temp;
        count++;
    }

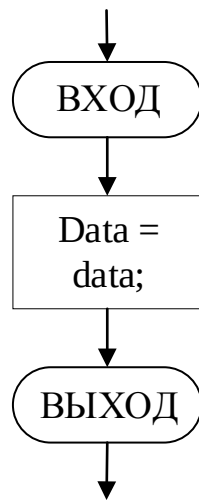
    private void button5_Click(object sender, EventArgs e)//очистка всех
полей
    {
        listBox.Items.Clear();
        listBox1.Items.Clear();
        listBox2.Items.Clear();
        listBox3.Items.Clear();
        numericUpDown.Value = 0;
        textBox1.Clear();
        textBox2.Clear();
        textBox3.Clear();
        textBox4.Clear();
        textBox5.Clear();
        textBox6.Clear();
        time.Reset();
    }
}
}

```

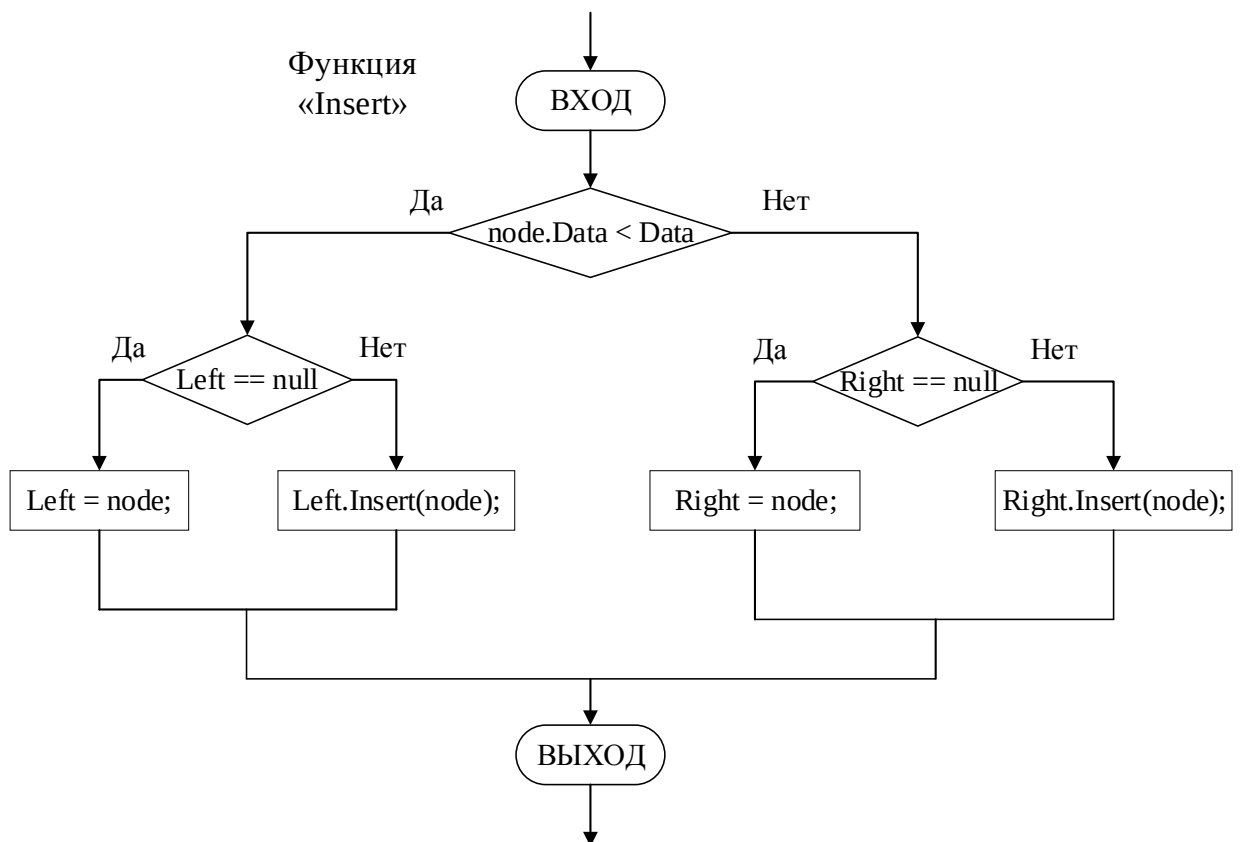
Приложение 2: Блок-схемы.

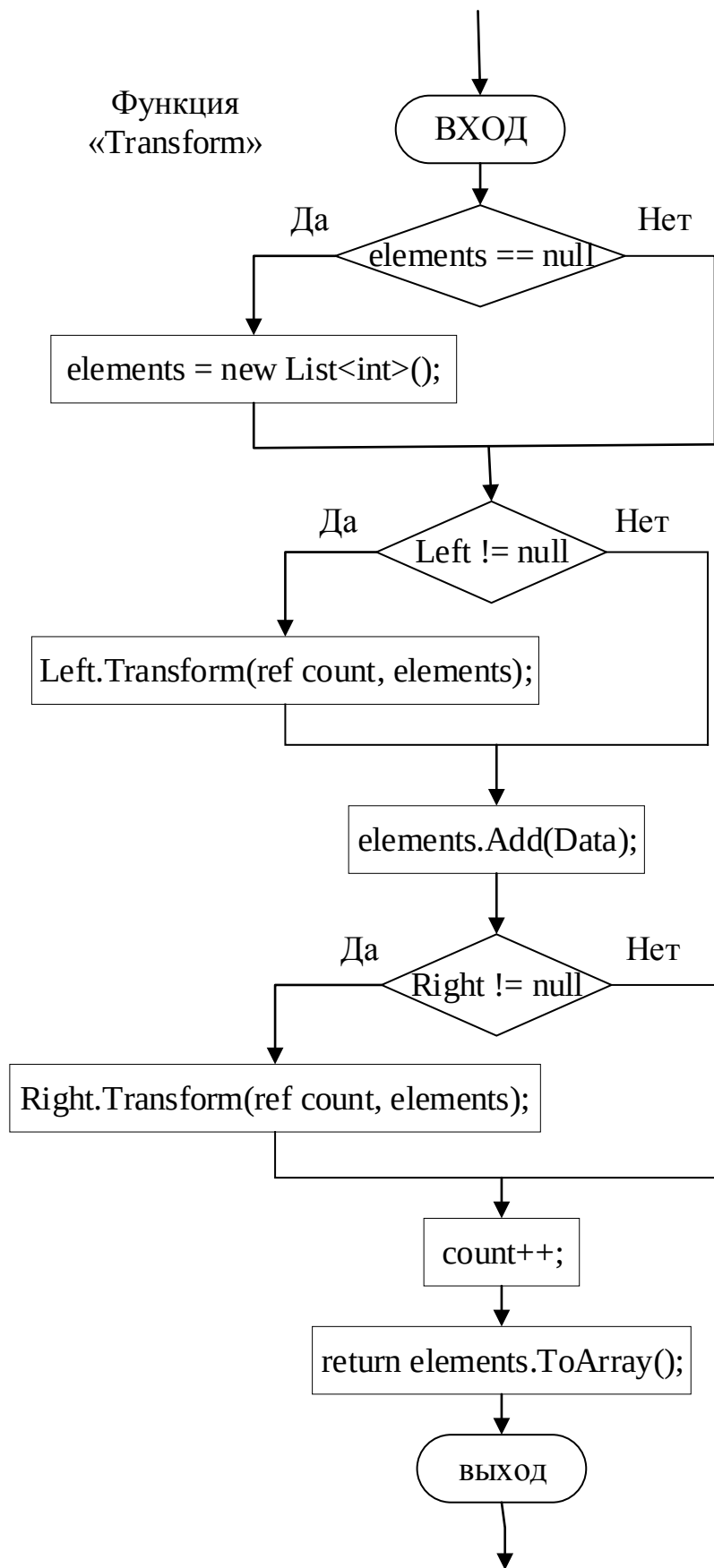


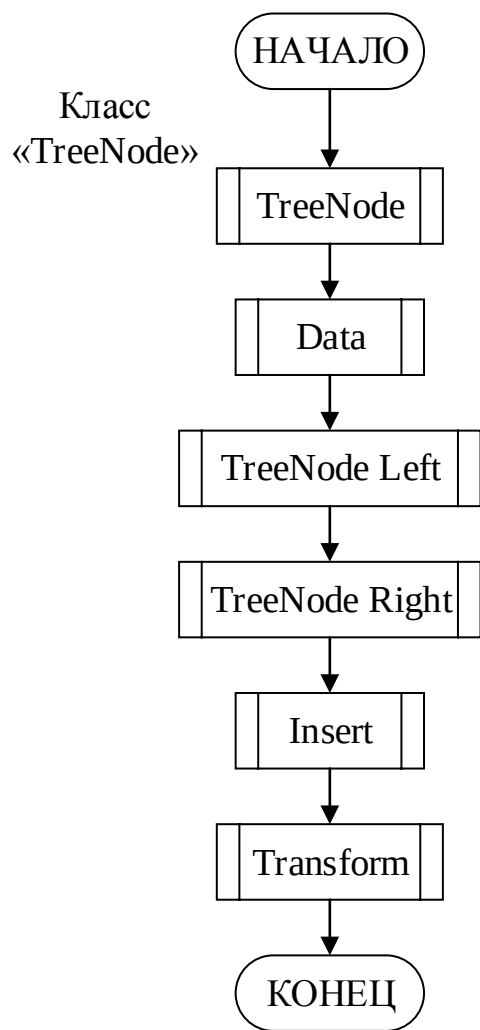
Функция
«TreeNode»

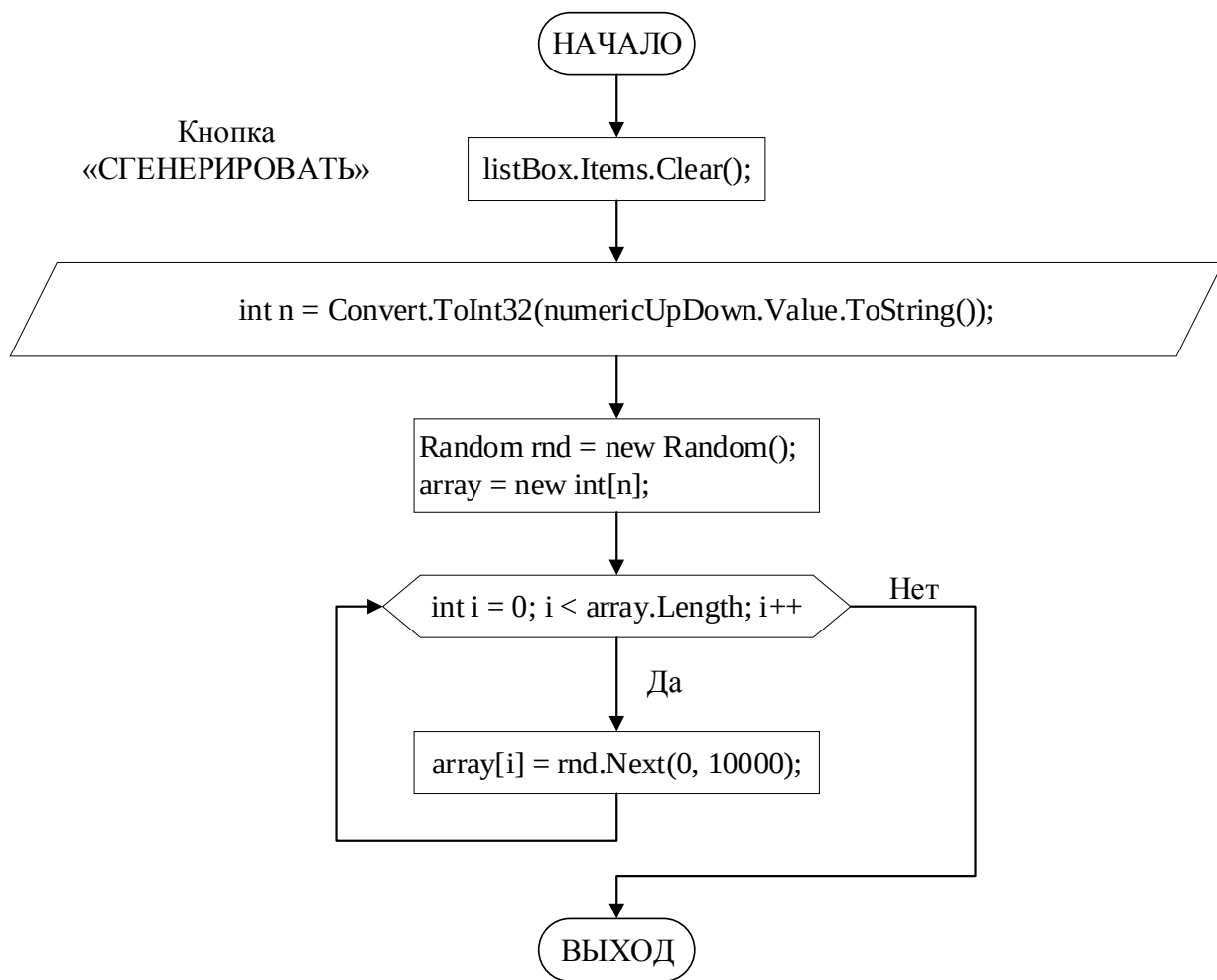


Функция
«Insert»

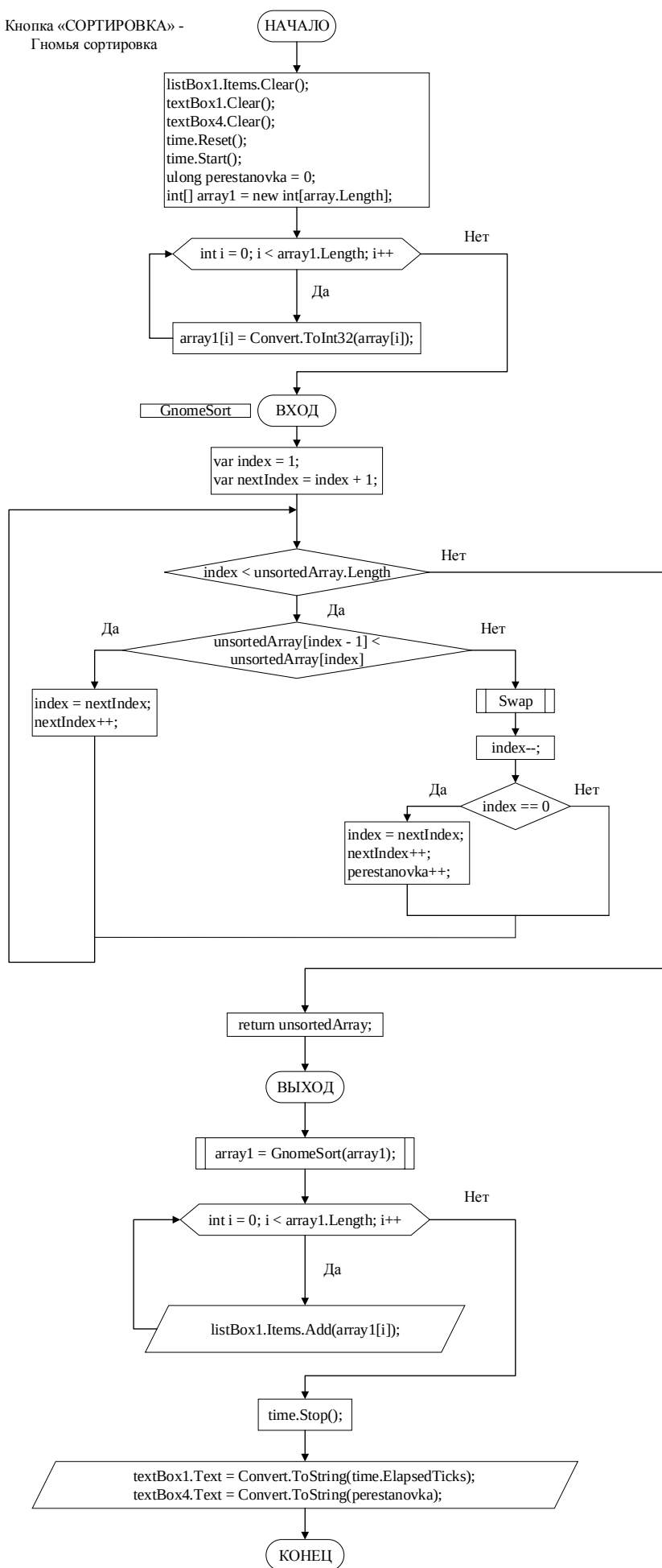




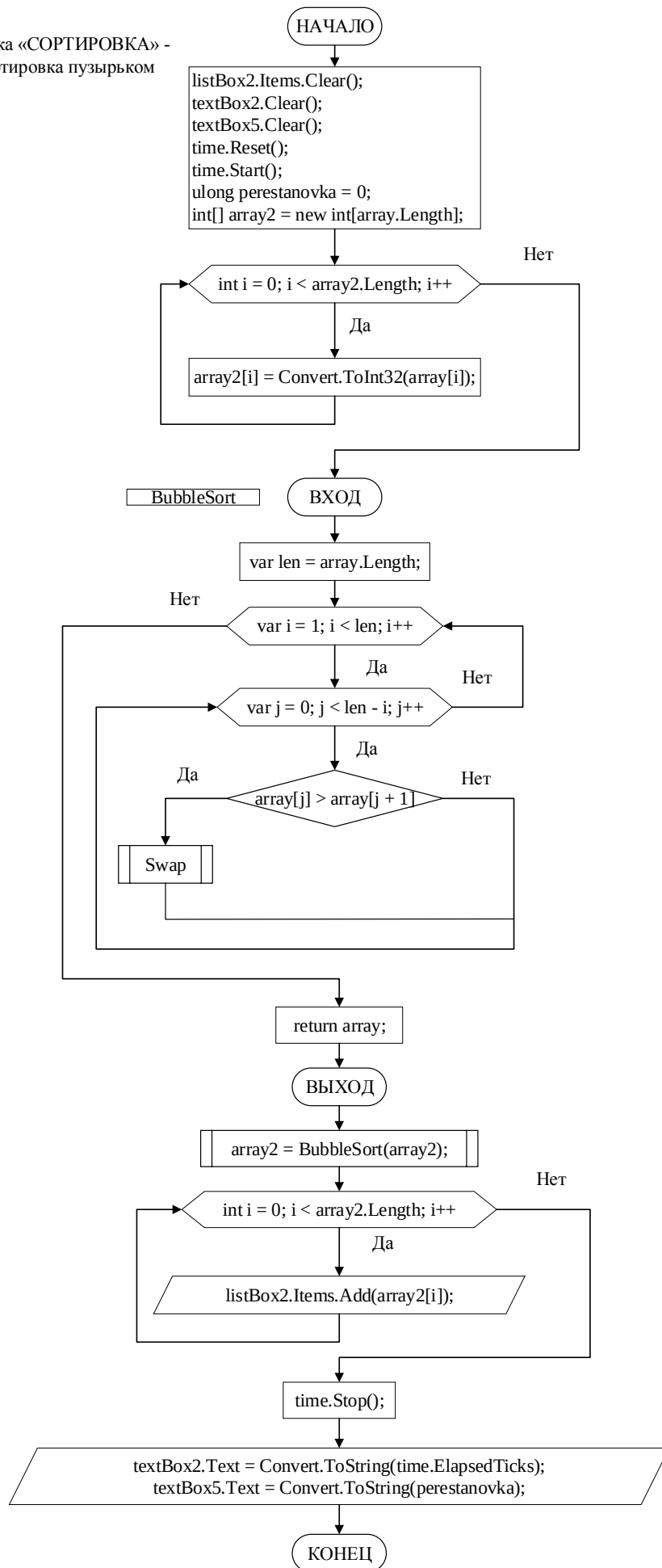




Кнопка «СОРТИРОВКА» -
Гномья сортировка



Кнопка «СОРТИРОВКА» -
Сортировка пузырьком



Кнопка «СОРТИРОВКА» -
Сортировка деревом

